LEZIONE 4 - Numeri Decimali in Binario e Rappresentazione Caratteri

Durata: 4 ore | Teoria: 2h | Esercizi: 2h

PARTE TEORICA (2 ore)

4.1 Rappresentazione dei Numeri con Virgola (30 min)

Finora abbiamo visto solo numeri interi. Ma come rappresentare numeri con la virgola (es. 3.14, -0.5, 123.456)?

Virgola Fissa vs Virgola Mobile

Due approcci principali:

1. **Fixed Point** (Virgola Fissa): la posizione della virgola è predefinita
2. **Floating Point** (Virgola Mobile): la virgola può "galleggiare" - maggiore flessibilità

4.2 Virgola Fissa (Fixed Point) (25 min)

Concetto Base

Nella virgola fissa, si decide a priori quanti bit dedicare alla parte intera e quanti alla parte decimale.

Esempio: Formato 8.8 (8 bit per parte intera, 8 bit per parte decimale)





Posizioni Decimali in Binario

Come nel sistema decimale, le posizioni dopo la virgola hanno pesi negativi:

Sistema Decimale:

$$123.45 = 1\times10^{2} + 2\times10^{1} + 3\times10^{0} + 4\times10^{-1} + 5\times10^{-2}$$
$$= 100 + 20 + 3 + 0.4 + 0.05$$

Sistema Binario:

Posizioni: 2^{2} 2^{1} 2^{0} . 2^{-1} 2^{-2} 2^{-3} 2^{-4}
Pesi: 4 2 1 . 0.5 0.25 0.125 0.0625

Tabella Potenze Negative di 2

Esponente	Potenza	Valore Decimale	Frazione
2^{-1}	1/2	0.5	1/2
2^{-2}	1/4	0.25	1/4
2^{-3}	1/8	0.125	1/8
2^{-4}	1/16	0.0625	1/16
2^{-5}	1/32	0.03125	1/32
2^{-6}	1/64	0.015625	1/64
2^{-7}	1/128	0.0078125	1/128
2^{-8}	1/256	0.00390625	1/256

Conversione Decimale → Binario (Parte Frazionaria)

Metodo delle Moltiplicazioni Successive:

1. Moltiplica la parte decimale per 2
2. Prendi la parte intera del risultato (0 o 1)
3. Usa solo la parte decimale per la moltiplicazione successiva
4. Ripeti finché la parte decimale diventa 0 o raggiungi il numero di bit desiderato
5. Leggi i bit **dall'alto verso il basso** (ordine di generazione)

Esempio 1: Convertire 0.75_{10} in binario

$0.75 \times 2 = 1.50 \rightarrow$ bit 1, continua con 0.50
$0.50 \times 2 = 1.00 \rightarrow$ bit 1, continua con 0.00
$0.00 \rightarrow$ STOP

Risultato: $0.75_{10} = 0.11_2$

Verifica: $0.11_2 = 1 \times 2^{-1} + 1 \times 2^{-2} = 0.5 + 0.25 = 0.75$ ✓

Esempio 2: Convertire 0.625_{10} in binario

$0.625 \times 2 = 1.25 \rightarrow$ bit 1, continua con 0.25
$0.25 \times 2 = 0.50 \rightarrow$ bit 0, continua con 0.50
$0.50 \times 2 = 1.00 \rightarrow$ bit 1, continua con 0.00
$0.00 \rightarrow$ STOP

Risultato: $0.625_{10} = 0.101_2$

Verifica: $0.101_2 = 1 \times 0.5 + 0 \times 0.25 + 1 \times 0.125 = 0.5 + 0.125 = 0.625$ ✓

Esempio 3: Convertire 0.1_{10} in binario (PROBLEMA!)

$0.1 \times 2 = 0.2 \rightarrow$ bit 0, continua con 0.2
$0.2 \times 2 = 0.4 \rightarrow$ bit 0, continua con 0.4
$0.4 \times 2 = 0.8 \rightarrow$ bit 0, continua con 0.8
$0.8 \times 2 = 1.6 \rightarrow$ bit 1, continua con 0.6

```
0.6 × 2 = 1.2  → bit 1, continua con 0.2
0.2 × 2 = 0.4  → bit 0, continua con 0.4  (CICLO!)
...

Risultato: 0.1₁₀ = 0.00011001100110011... (periodico!)
```

IMPORTANTE: Non tutti i numeri decimali hanno una rappresentazione binaria finita!

- 0.5, 0.25, 0.125, 0.75, 0.625 → rappresentazione finita
- 0.1, 0.2, 0.3, 0.6, 0.7 → rappresentazione periodica infinita

Questo causa **errori di arrotondamento** nei calcoli con virgola mobile!

Conversione Binario → Decimale (Numeri con Virgola)

Metodo: Somma dei bit moltiplicati per i loro pesi (potenze negative di 2)

Esempio 1: Convertire 101.011₂ in decimale

```
 1   0   1 . 0   1   1
 ↓   ↓   ↓   ↓   ↓   ↓
 4   2   1   0.5 0.25 0.125

Calcolo:
= 1×4 + 0×2 + 1×1 + 0×0.5 + 1×0.25 + 1×0.125
= 4 + 0 + 1 + 0 + 0.25 + 0.125
= 5.375₁₀
```

Esempio 2: Convertire 11.11₂ in decimale

```
 1   1 . 1   1
 ↓   ↓   ↓   ↓
 2   1   0.5 0.25

= 1×2 + 1×1 + 1×0.5 + 1×0.25
= 2 + 1 + 0.5 + 0.25
= 3.75₁₀
```

Conversione Numero Completo (Parte Intera + Decimale)

Esempio: Convertire 13.625_{10} in binario

Passo 1 - Parte intera (13):

```
13 ÷ 2 = 6  resto 1 ←┐
 6 ÷ 2 = 3  resto 0  |
 3 ÷ 2 = 1  resto 1  |
 1 ÷ 2 = 0  resto 1 ←┘

Parte intera: 1101₂
```

Passo 2 - Parte decimale (0.625):

```
0.625 × 2 = 1.25  → bit 1
0.25  × 2 = 0.50  → bit 0
0.50  × 2 = 1.00  → bit 1

Parte decimale: .101₂
```

Risultato: $13.625_{10} = 1101.101_2$

Verifica:

```
1101.101₂ = 8 + 4 + 0 + 1 + 0.5 + 0 + 0.125
          = 13 + 0.625
          = 13.625₁₀ ✓
```

Limitazioni della Virgola Fissa

Vantaggi:

- Veloce
- Prevedibile
- Semplice da implementare

Svantaggi:

- Range limitato
- Precisione fissa (non può rappresentare sia numeri molto grandi che molto piccoli)
- Spreco di bit se i numeri sono molto diversi tra loro

Esempio con formato 8.8:

```
Range: da 0.00390625 a 255.99609375
Precisione: 1/256 ≈ 0.004

Non può rappresentare:
- 1000.5 (troppo grande)
- 0.0001 (troppo piccolo)
```

4.3 Virgola Mobile (Floating Point) (35 min)

Notazione Scientifica

Prima di capire il floating point, ricordiamo la **notazione scientifica**:

Formato: ±mantissa × 10^esponente

Esempi:

```
1,234,000 = 1.234 × 10⁶
0.000056 = 5.6 × 10⁻⁵
-299.8 = -2.998 × 10²
```

Il floating point binario usa lo stesso concetto!

Standard IEEE 754

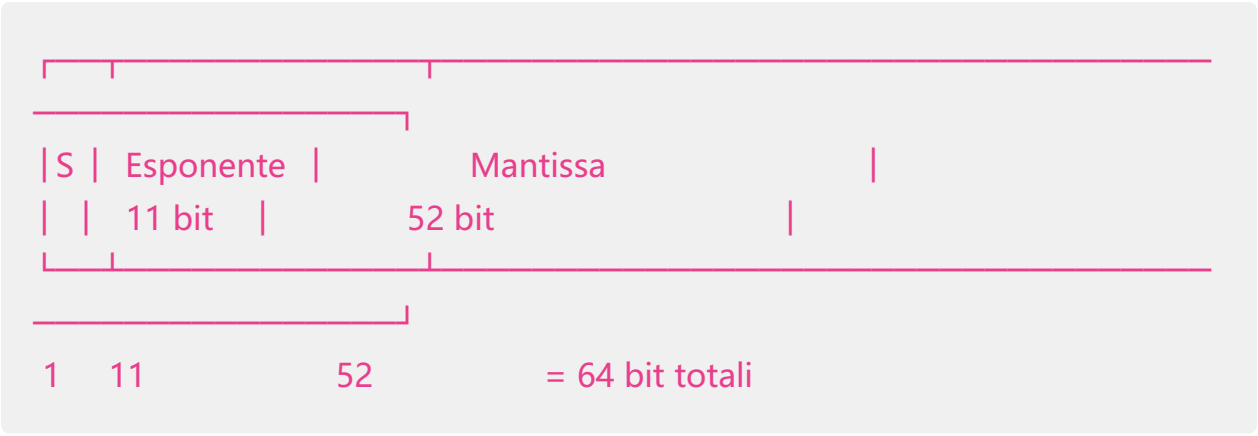
Lo **standard IEEE 754** (1985) definisce come rappresentare i numeri in virgola mobile nei computer. È universalmente adottato.

Due formati principali:

1. Singola Precisione (32 bit) - `float` in C



2. Doppia Precisione (64 bit) - `double` in C



Componenti di un Numero Floating Point

1. SEGNO (Sign - S): 1 bit

- 0 = positivo
- 1 = negativo

2. ESPONENTE (Exponent - E): 8 bit (float) o 11 bit (double)

- Rappresentato con **Eccesso-127** (float) o **Eccesso-1023** (double)
- Permette esponenti positivi e negativi

3. MANTISSA (Significand/Fraction - M): 23 bit (float) o 52 bit (double)

- Rappresenta le cifre significative del numero
- **Bit nascosto (hidden bit):** il primo bit (sempre 1) non viene memorizzato!

Formato del Numero

Formula generale:

```
Valore = (-1)^S × (1.M) × 2^(E-bias)

Dove:
- S = bit di segno
- M = mantissa (bits frazionari)
- E = esponente (come memorizzato)
- bias = 127 (float) o 1023 (double)
```

Il "1." nascosto: In formato normalizzato, la mantissa è sempre tra 1.0 e 2.0, quindi il "1" iniziale non viene memorizzato (guadagno di 1 bit di precisione!)

Esempio Completo: Float 32 bit

Rappresentare 12.375_{10} in IEEE 754 singola precisione

Passo 1: Converti in binario

Parte intera: $12_{10} = 1100_2$
Parte decimale: $0.375_{10} = 0.011_2$

$12.375_{10} = 1100.011_2$

Passo 2: Normalizza in forma 1.xxx × 2^n

$1100.011_2 = 1.100011 \times 2^3$

(Sposta la virgola a sinistra di 3 posizioni)

Passo 3: Determina i componenti

Segno:

12.375 è positivo → S = 0

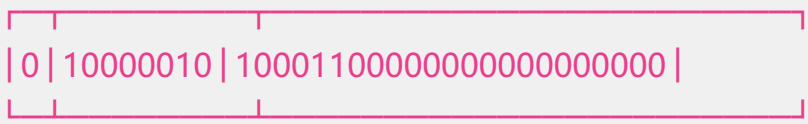
Esponente:

Esponente reale: 3
Esponente con bias: 3 + 127 = 130_{10} = 10000010_2

Mantissa:

Parte dopo "1.": 100011
Completata a 23 bit: 10001100000000000000000
(aggiungi zeri a destra)

Passo 4: Assembla il numero



In esadecimale: 0x41460000

Verifica:

S = 0 (positivo)
E = 10000010_2 = 130_{10}
Esponente reale = 130 - 127 = 3

Mantissa = 1.100011_2 = 1 + 0.5 + 0 + 0 + 0.0625 + 0.03125
= 1.59375

Valore = $+1 \times 1.59375 \times 2^3$
= 1.59375×8
= 12.75... ERRORE!

Ops! Ho fatto un errore. Correggiamo:

0.375 in binario:

```
0.375 × 2 = 0.75  → 0
0.75  × 2 = 1.5   → 1
0.5   × 2 = 1.0   → 1

0.375₁₀ = 0.011₂  ✓
```

Quindi $12.375_{10} = 1100.011_2 = 1.100011 \times 2^3$ ✓

Mantissa dopo "1.": 100011… (solo 6 bit significativi)

Verifica:

$$1.100011_2 = 1 + 1\times2^{-1} + 0\times2^{-2} + 0\times2^{-3} + 0\times2^{-4} + 1\times2^{-5} + 1\times2^{-6}$$
$$= 1 + 0.5 + 0.03125 + 0.015625$$
$$= 1.546875$$

Valore = $1.546875 \times 2^3 = 1.546875 \times 8 = 12.375$ ✓

Casi Speciali in IEEE 754

1. Zero

```
Esponente = 0
Mantissa = 0

Nota: esistono +0 e -0 (differiscono solo nel bit di segno)
```

2. Infinito

```
Esponente = tutti 1 (255 per float)
Mantissa = 0

+∞: S=0, E=11111111, M=0
-∞: S=1, E=11111111, M=0
```

3. NaN (Not a Number)

Esponente = tutti 1 (255 per float)
Mantissa ≠ 0

Esempi di operazioni che producono NaN:
- 0 / 0
- ∞ - ∞
- √(-1)

4. Numeri Denormalizzati (Subnormal)

Esponente = 0
Mantissa ≠ 0

Usati per numeri molto piccoli, vicini a zero
Formula: (-1)^S × (0.M) × 2^(-126)

Range e Precisione

Float (32 bit):

Range: $\pm 1.18 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$
Precisione: circa 7 cifre decimali
Epsilon (differenza minima): ~1.19×10^{-7}

Double (64 bit):

Range: $\pm 2.23 \times 10^{-308}$ a $\pm 1.8 \times 10^{308}$
Precisione: circa 15-16 cifre decimali
Epsilon: ~2.22×10^{-16}

Problemi del Floating Point

1. Errori di Arrotondamento

```
float a = 0.1f;
float b = 0.2f;
float c = a + b;
// c potrebbe NON essere esattamente 0.3!
// c ≈ 0.30000001 (esempio)
```

2. Perdita di Precisione con Numeri Molto Diversi

```
float grande = 1000000.0f;
float piccolo = 0.000001f;
float somma = grande + piccolo;
// somma == grande (piccolo viene "perso")
```

3. Confronti Pericolosi

```
// MALE!
if (x == 0.3) { ... }

// BENE!
if (fabs(x - 0.3) < 0.00001) { ... }
```

4. Operazioni Non Associative

```
(a + b) + c ≠ a + (b + c)  // Possibile!
```

4.4 Caratteri e Codifica ASCII (30 min)

Cos'è un Carattere?

Un **carattere** è un simbolo (lettera, numero, punteggiatura, simbolo speciale) che rappresentiamo con un codice numerico.

IMPORTANTE: Il numero 5 e il carattere ‘5’ sono diversi!

- Numero 5: valore numerico, su cui fare calcoli
- Carattere ‘5’: simbolo da visualizzare, codice 53 in ASCII

ASCII - American Standard Code for Information Interchange

Storia: Sviluppato negli anni ’60 per standardizzare la rappresentazione dei caratteri.

Caratteristiche:

- Usa **7 bit** (128 caratteri: 0-127)
- Spesso memorizzato in **8 bit** (1 byte), con l’8° bit inutilizzato o usato per parità
- Standard universale per caratteri inglesi di base

Struttura della Tabella ASCII

Divisione in gruppi:

0-31: Caratteri di Controllo (non stampabili)

```
0   = NUL (Null)
7   = BEL (Bell/campanello)
8   = BS  (Backspace)
9   = TAB (Tab orizzontale)
10  = LF  (Line Feed / new line)
13  = CR  (Carriage Return)
27  = ESC (Escape)
```

32-47: Spazio e Simboli

```
32  = spazio
33  = !
...
47  = /
```

48-57: Cifre Numeriche

```
48  = '0'
49  = '1'
...
57  = '9'
```

58-64: Simboli

```
58  = :
...
64  = @
```

65-90: Lettere Maiuscole

```
65  = 'A'
66  = 'B'
...
90  = 'Z'
```

91-96: Simboli

```
91  = [
...
96  = `
```

97-122: Lettere Minuscole

```
97  = 'a'
98  = 'b'
...
122 = 'z'
```

123-126: Simboli

```
123 = {
...
126 = ~
```

127: DEL (Delete)

Tabella ASCII Completa (Caratteri Principali)

Dec	Hex	Binario	Char	Descrizione
0	00	00000000	NUL	Null
9	09	00001001	TAB	Tab
10	0A	00001010	LF	Line Feed
13	0D	00001101	CR	Carriage Return
32	20	00100000	' '	Spazio
48	30	00110000	'0'	Zero
49	31	00110001	'1'	Uno
57	39	00111001	'9'	Nove
65	41	01000001	'A'	A maiuscola
66	42	01000010	'B'	B maiuscola
90	5A	01011010	'Z'	Z maiuscola
97	61	01100001	'a'	a minuscola
98	62	01100010	'b'	b minuscola
122	7A	01111010	'z'	z minuscola

Relazioni Utili in ASCII

1. Maiuscole ↔ Minuscole:

```
Differenza = 32 (0x20)

'A' (65) + 32 = 'a' (97)
'Z' (90) + 32 = 'z' (122)

Per convertire:
maiuscola → minuscola: aggiungi 32 (o OR con 0x20)
minuscola → maiuscola: sottrai 32 (o AND con ~0x20)
```

2. Carattere Cifra → Valore Numerico:

```
'0' = 48
'1' = 49
...
'9' = 57

Per convertire:
Carattere → numero: sottrai 48 (o '0')
'5' - '0' = 53 - 48 = 5

Numero → carattere: aggiungi 48 (o '0')
5 + '0' = 5 + 48 = 53 = '5'
```

3. Bit di Differenza Maiuscola/Minuscola:

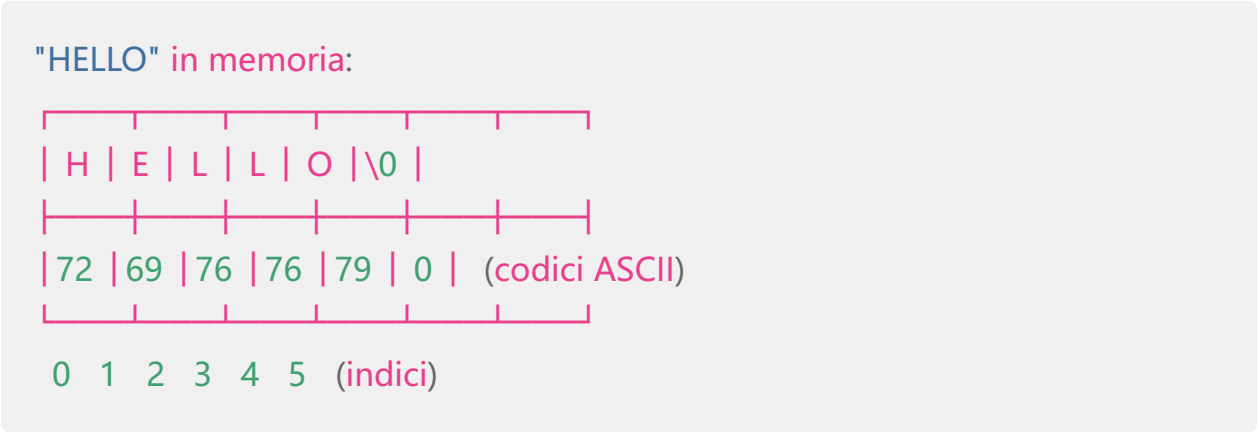
```
Binario:
'A' = 01000001
'a' = 01100001
       ↑
Solo il 6° bit (da destra, partendo da 0) cambia!

Bit 5 (0x20) = differenza maiuscola/minuscola
```

Codifica di Stringhe

Una **stringa** è una sequenza di caratteri.

In C, le stringhe sono array di caratteri terminati con \0 (NULL, codice 0):



Lunghezza: 5 caratteri visibili + 1 terminatore = 6 byte totali

Limitazioni di ASCII

Problema: ASCII supporta solo caratteri inglesi di base!

Non può rappresentare:

- Lettere accentate (à, è, ñ, ü)
- Alfabeti non latini (greco, cirillico, arabo, cinese, giapponese, ecc.)
- Simboli matematici avanzati
- Emoji 😀

Soluzione: Charset estesi!

4.5 Charset Estesi: Unicode e UTF-8 (30 min)

ASCII Esteso (8 bit)

Extended ASCII: Usa tutti gli 8 bit (256 caratteri: 0-255)

Codici 128-255 usati per:

- Lettere accentate
- Simboli matematici

- Box drawing characters
- Simboli valuta (€, £, ¥)

Problema: Molti standard incompatibili!

- ISO 8859-1 (Latin-1): Europa occidentale
- Windows-1252: Variante Microsoft
- ISO 8859-5: Cirillico
- …

Non esiste un modo universale per rappresentare tutti i linguaggi!

Unicode - La Soluzione Universale

Unicode è uno standard che assegna un codice univoco (chiamato **code point**) a ogni carattere di ogni lingua del mondo.

Versione attuale: Unicode 15.0 (2022)

- Oltre **149,000 caratteri**
- Supporta più di 150 scritture
- Include emoji, simboli matematici, caratteri storici

Formato Code Point: U+XXXX (esadecimale)

Esempi:

```
U+0041 = 'A'
U+00E9 = 'é'
U+03B1 = 'α' (alfa greco)
U+4E2D = '中' (cinese)
U+1F600 = '😀' (emoji)
```

Unicode NON è una codifica! È solo un'assegnazione di numeri ai caratteri.

Per memorizzare Unicode serve una **codifica**: UTF-8, UTF-16, UTF-32.

UTF-8 - Codifica Universale

UTF-8 (Unicode Transformation Format - 8 bit) è la codifica Unicode più usata su Internet.

Caratteristiche:

- **Lunghezza variabile**: da 1 a 4 byte per carattere
- **Compatibile con ASCII**: primi 128 caratteri = ASCII
- **Auto-sincronizzante**: si capisce dove inizia un carattere
- **Efficiente**: caratteri comuni usano meno byte

Schema di Codifica UTF-8

1 byte (0-127): caratteri ASCII

```
0xxxxxxx
Esempio: 'A' = U+0041 = 0x41 = 01000001
```

2 byte (128-2047): alfabeti latini estesi, greco, cirillico, arabo, ebraico

```
110xxxxx 10xxxxxx
Esempio: 'é' = U+00E9 = 11000011 10101001 = 0xC3 0xA9
```

3 byte (2048-65535): la maggior parte degli altri alfabeti (cinese, giapponese, coreano)

```
1110xxxx 10xxxxxx 10xxxxxx
Esempio: '中' = U+4E2D = 11100100 10111000 10101101
```

4 byte (65536-1114111): emoji, caratteri rari, simboli matematici avanzati

```
11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
Esempio: '😀' = U+1F600 = 11110000 10011111 10011000 10000000
```

Esempio: Codifica “Hello, 世界!” in UTF-8

```
Carattere  | Code Point | UTF-8 (hex)     | Byte
-----------|------------|-----------------|------
'H'        | U+0048     | 48              | 1
'e'        | U+0065     | 65              | 1
```

```
'l'      | U+006C    | 6C          | 1
'l'      | U+006C    | 6C          | 1
'o'      | U+006F    | 6F          | 1
','      | U+002C    | 2C          | 1
' '      | U+0020    | 20          | 1
'世'     | U+4E16    | E4 B8 96    | 3
'界'     | U+754C    | E7 95 8C    | 3
'!'      | U+0021    | 21          | 1

Totale: 14 byte
```

UTF-16 e UTF-32

UTF-16:

- Usa 2 o 4 byte per carattere
- Caratteri comuni (BMP): 2 byte
- Caratteri rari: 4 byte (coppia surrogata)
- Usato internamente da Windows, Java, JavaScript

UTF-32:

- Usa sempre 4 byte per carattere
- Semplice ma spreca spazio
- Raramente usato

Confronto Codifiche

Codifica	Byte per char	Range	Pro	Contro
ASCII	1	128 car.	Semplice, veloce	Solo inglese
ASCII Esteso	1	256 car.	Compatto	Incompatibilità
UTF-8	1-4	1.1M+ car.	Compatibile ASCII, efficiente	Lunghezza variabile

Codifica	Byte per char	Range	Pro	Contro
UTF-16	2-4	1.1M+ car.	Efficiente per lingue asiatiche	Non compatibile ASCII
UTF-32	4	1.1M+ car.	Accesso diretto	Spreco di spazio

BOM (Byte Order Mark)

BOM è una sequenza speciale all'inizio di un file per indicare:

- Quale codifica UTF viene usata
- L'ordine dei byte (big-endian o little-endian per UTF-16/32)

```
UTF-8: EF BB BF
UTF-16 BE: FE FF
UTF-16 LE: FF FE
UTF-32 BE: 00 00 FE FF
UTF-32 LE: FF FE 00 00
```

4.6 Differenza tra Numero e Carattere (10 min)

Questa è una distinzione **fondamentale** in programmazione!

Numero vs Carattere

Il numero 5:

```
Tipo: int
Valore binario: 00000101 (se 8 bit)
Uso: calcoli matematici (5 + 3 = 8)
In memoria: rappresentazione binaria diretta
```

Il carattere '5':

```
Tipo: char
Codice ASCII: 53 (0x35)
Valore binario: 00110101
Uso: visualizzazione, testo
In memoria: codice ASCII del simbolo '5'
```

Esempi Pratici

In C:

```c
int numero = 5;       // Valore numerico 5
char carattere = '5';  // Codice ASCII 53

printf("%d\n", numero);     // Stampa: 5
printf("%d\n", carattere);  // Stampa: 53 (codice ASCII)
printf("%c\n", carattere);  // Stampa: 5 (il simbolo)
```

Operazioni:

```c
int a = 5;
int b = 3;
int somma = a + b;     // somma = 8 ✓

char c1 = '5';        // ASCII 53
char c2 = '3';        // ASCII 51
int somma2 = c1 + c2;  // somma2 = 104 (53+51) ✗ NON è 8!

// Per convertire:
int valore1 = c1 - '0';  // valore1 = 53 - 48 = 5 ✓
int valore2 = c2 - '0';  // valore2 = 51 - 48 = 3 ✓
int somma3 = valore1 + valore2;  // somma3 = 8 ✓
```

Array di Caratteri vs Array di Numeri

Stringa “123”:

```
char stringa[] = "123";

In memoria:
┌─────┬─────┬─────┬─────┐
| 49  | 50  | 51  | 0   |  (codici ASCII + terminatore)
└─────┴─────┴─────┴─────┘
 '1'  '2'  '3'  '\0'

Lunghezza: 4 byte
```

Array di numeri [1, 2, 3]:

```
int numeri[] = {1, 2, 3};

In memoria (assumendo int = 4 byte):
┌─────────┬─────────┬─────────┐
|    1    |    2    |    3    |  (valori interi)
└─────────┴─────────┴─────────┘
0x00000001 0x00000002 0x00000003

Lunghezza: 12 byte
```

Conversioni Comuni

Carattere cifra → Numero:

```
char c = '7';
int n = c - '0';    // n = 7

// Oppure
int n = c - 48;     // n = 7
```

Numero → Carattere cifra:

```
int n = 7;
char c = n + '0';    // c = '7'

// Oppure
char c = n + 48;     // c = '7'
```

Stringa → Numero:

```
char str[] = "123";
int num = atoi(str);      // num = 123 (funzione stdlib.h)

// Manualmente:
int num = 0;
for (int i = 0; str[i] != '\0'; i++) {
    num = num * 10 + (str[i] - '0');
}
// num = 0*10 + 1 = 1
// num = 1*10 + 2 = 12
// num = 12*10 + 3 = 123
```

Numero → Stringa:

```
int num = 123;
char str[20];
sprintf(str, "%d", num);  // str = "123"

// O usa itoa() se disponibile
```

PARTE PRATICA (2 ore)

ESERCIZI GUIDATI

Esercizio 1: Numeri con Virgola - Conversioni (25 min)

1.1 Converti in decimale i seguenti numeri binari con virgola:

1. 110.101_2

Soluzione:

```
 1   1   0 . 1   0   1
 ↓   ↓   ↓   ↓   ↓   ↓
 4   2   1  0.5 0.25 0.125
```

$= 1\times4 + 1\times2 + 0\times1 + 1\times0.5 + 0\times0.25 + 1\times0.125$
$= 4 + 2 + 0 + 0.5 + 0 + 0.125$
$= 6.625_{10}$

1. 1.001_2

Soluzione:

```
 1 . 0   0   1
 ↓   ↓   ↓   ↓
 1  0.5 0.25 0.125
```

$= 1\times1 + 0\times0.5 + 0\times0.25 + 1\times0.125$
$= 1 + 0.125$
$= 1.125_{10}$

1. 101.11_2

Soluzione:

```
 1   0   1 . 1   1
 ↓   ↓   ↓   ↓   ↓
```

4 2 1 0.5 0.25

= 1×4 + 0×2 + 1×1 + 1×0.5 + 1×0.25
= 4 + 1 + 0.5 + 0.25
= 5.75_{10}

1.2 Converti in binario (con massimo 8 bit decimali):

1. 3.5_{10}

Soluzione:
Parte intera: $3_{10} = 11_2$

Parte decimale: 0.5
0.5 × 2 = 1.0 → bit 1, STOP

Risultato: $3.5_{10} = 11.1_2$

Verifica: $11.1_2 = 2 + 1 + 0.5 = 3.5$ ✓

1. 5.25_{10}

Soluzione:
Parte intera: $5_{10} = 101_2$

Parte decimale: 0.25
0.25 × 2 = 0.5 → bit 0
0.5 × 2 = 1.0 → bit 1, STOP

Risultato: $5.25_{10} = 101.01_2$

Verifica: $101.01_2 = 4 + 1 + 0.25 = 5.25$ ✓

1. 7.875_{10}

Soluzione:
Parte intera: $7_{10} = 111_2$

Parte decimale: 0.875
0.875 × 2 = 1.75 → bit 1
0.75 × 2 = 1.5 → bit 1
0.5 × 2 = 1.0 → bit 1, STOP

Risultato: $7.875_{10} = 111.111_2$

Verifica: $111.111_2 = 4 + 2 + 1 + 0.5 + 0.25 + 0.125 = 7.875$ ✓

1. 0.1_{10} (mostra il problema!)

Soluzione:
0.1 × 2 = 0.2 → bit 0
0.2 × 2 = 0.4 → bit 0
0.4 × 2 = 0.8 → bit 0
0.8 × 2 = 1.6 → bit 1
0.6 × 2 = 1.2 → bit 1
0.2 × 2 = 0.4 → bit 0 (si ripete!)
0.4 × 2 = 0.8 → bit 0
0.8 × 2 = 1.6 → bit 1

Risultato: $0.1_{10} = 0.00011001100110011...$ (periodico!)

Con 8 bit: 0.00011001_2
Valore approssimato: 0.09765625_{10}

Errore: 0.1 - 0.09765625 = 0.00234375

1.3 Dimostra che $0.3 \neq 0.1 + 0.2$ in binario (con 8 bit decimali):

Soluzione:
$0.1_{10} \approx 0.00011001_2$ (troncato a 8 bit)

$0.2_{10} \approx 0.00110011_2$ (troncato a 8 bit)

Somma in binario:

```
  0.00011001
+ 0.00110011
------------
  0.01001100₂
```

Converti in decimale:
$0.01001100_2 = 0.25 + 0.0625 + 0.015625 = 0.328125_{10}$

Ma 0.3 in binario (8 bit):
$0.3 \times 2 = 0.6 \rightarrow 0$
$0.6 \times 2 = 1.2 \rightarrow 1$
$0.2 \times 2 = 0.4 \rightarrow 0$
$0.4 \times 2 = 0.8 \rightarrow 0$
$0.8 \times 2 = 1.6 \rightarrow 1$
$0.6 \times 2 = 1.2 \rightarrow 1$ (ciclo)
...

$0.3_{10} \approx 0.01001100_2$ (troncato)

Ma attendevi 0.3, ottieni 0.328125!
Ecco perché 0.1 + 0.2 ≠ 0.3 nei computer!

Esercizio 2: IEEE 754 Float (30 min)

2.1 Analizza il seguente numero IEEE 754 (32 bit):

```
0 10000010 10100000000000000000000
```

Soluzione:
Segno: 0 → positivo

Esponente: $10000010_2 = 130_{10}$

Esponente reale: 130 - 127 = 3

Mantissa: 10100000000000000000000
Con bit nascosto: 1.101_2
$= 1 + 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3}$
$= 1 + 0.5 + 0.125$
$= 1.625$

Valore: $(+1) \times 1.625 \times 2^3$
$= 1.625 \times 8$
$= 13.0_{10}$

2.2 Rappresenta -6.75_{10} in IEEE 754 (32 bit):

Soluzione:
Passo 1: Converti in binario
$6_{10} = 110_2$
0.75_{10}:
$0.75 \times 2 = 1.5 \rightarrow 1$
$0.5 \times 2 = 1.0 \rightarrow 1$

$6.75_{10} = 110.11_2$

Passo 2: Normalizza
$110.11_2 = 1.1011 \times 2^2$

Passo 3: Componenti
Segno: negativo → S = 1
Esponente: $2 + 127 = 129_{10} = 10000001_2$
Mantissa: 1011 (dopo "1.")
Completa a 23 bit: 10110000000000000000000

Passo 4: Risultato

1	10000001	10110000000000000000000

In esadecimale: 0xC0D80000

2.3 Identifica questi valori speciali IEEE 754:

1. `0 11111111 00000000000000000000000`

Soluzione:
S = 0
E = 11111111 (tutti 1)
M = 0

Valore: +∞ (infinito positivo)

1. `1 11111111 00000000000000000000000`

Soluzione:
S = 1
E = 11111111 (tutti 1)
M = 0

Valore: -∞ (infinito negativo)

1. `0 11111111 10000000000000000000000`

Soluzione:
S = 0
E = 11111111 (tutti 1)
M ≠ 0

Valore: NaN (Not a Number)

1. `0 00000000 00000000000000000000000`

```
Soluzione:
S = 0
E = 0
M = 0

Valore: +0 (zero positivo)
```

Esercizio 3: ASCII e Caratteri (25 min)

3.1 Converti le seguenti stringhe in codici ASCII (decimali):

1. “ABC”

```
Soluzione:
'A' = 65
'B' = 66
'C' = 67

Sequenza: 65 66 67
```

1. “Hello”

```
Soluzione:
'H' = 72
'e' = 101
'l' = 108
'l' = 108
'o' = 111

Sequenza: 72 101 108 108 111
```

1. “123”

Soluzione:
'1' = 49
'2' = 50
'3' = 51

Sequenza: 49 50 51

ATTENZIONE: Non è il numero 123!

3.2 Converti i seguenti codici ASCII in caratteri:

1. 65 66 67

Soluzione: "ABC"

1. 72 105 33

Soluzione:
72 = 'H'
105 = 'i'
33 = '!'

Stringa: "Hi!"

1. 48 49 50

Soluzione:
48 = '0'
49 = '1'
50 = '2'

Stringa: "012"

3.3 Conversione maiuscole/minuscole:

1. Converti ‘A’ in ‘a’

```
Soluzione:
'A' = 65
Differenza = 32
'a' = 65 + 32 = 97 ✓

In binario:
'A' = 01000001
'a' = 01100001
      ↑
Solo il bit 5 cambia!

Metodo veloce: 'A' | 0x20 = 'a'
```

1. Converti ‘z’ in ‘Z’

```
Soluzione:
'z' = 122
Differenza = -32
'Z' = 122 - 32 = 90 ✓

Metodo veloce: 'z' & ~0x20 = 'Z'
```

3.4 Conversione carattere ↔ numero:

1. Converti ‘7’ in numero 7

```
Soluzione:
'7' = 55 (ASCII)
'0' = 48 (ASCII)
```

numero = '7' - '0' = 55 - 48 = 7 ✓

1. Converti numero 9 in carattere ‘9’

Soluzione:
numero = 9
'0' = 48

carattere = 9 + '0' = 9 + 48 = 57 = '9' ✓

1. Converti la stringa “42” nel numero 42

Soluzione:
'4' = 52, valore = 52 - 48 = 4
'2' = 50, valore = 50 - 48 = 2

numero = 4 × 10 + 2 = 42 ✓

Esercizio 4: UTF-8 (20 min)

4.1 Determina quanti byte servono per codificare in UTF-8:

1. ‘A’ (U+0041)

Soluzione:
U+0041 = 65_{10} < 128

Range 0-127: 1 byte
Codifica: 0x41 = 01000001

1. ‘è’ (U+00E8)

Soluzione:
U+00E8 = 232_{10}

Range 128-2047: 2 byte
Formato: 110xxxxx 10xxxxxx

Calcolo:
$232_{10} = 11101000_2$

Dividi in bit:
11 101000
↓↓ ↓↓↓↓↓↓
110(00011) 10(101000)

Codifica: 0xC3 0xA8 = 11000011 10101000

1. '€' (U+20AC)

Soluzione:
$U+20AC = 8364_{10}$

Range 2048-65535: 3 byte
Formato: 1110xxxx 10xxxxxx 10xxxxxx

$8364_{10} = 0010000010101100_2$
$= 0010\ 000010\ 101100$

1110(0010) 10(000010) 10(101100)

Codifica: 0xE2 0x82 0xAC = 11100010 10000010 10101100

1. '😀' (U+1F600)

Soluzione:
$U+1F600 = 128512_{10}$

Range 65536-1114111: 4 byte
Formato: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

$128512_{10} = 11111011000000000_2$
$= 000\ 011111\ 011000\ 000000$

11110(000) 10(011111) 10(011000) 10(000000)

Codifica: 0xF0 0x9F 0x98 0x80

4.2 Calcola la dimensione in byte della stringa UTF-8:

"Hello, 世界!"

Soluzione:
'H' = U+0048 → 1 byte
'e' = U+0065 → 1 byte
'l' = U+006C → 1 byte
'l' = U+006C → 1 byte
'o' = U+006F → 1 byte
',' = U+002C → 1 byte
' ' = U+0020 → 1 byte
'世' = U+4E16 → 3 byte (CJK)
'界' = U+754C → 3 byte (CJK)
'!' = U+0021 → 1 byte

Totale: 7 + 3 + 3 + 1 = 14 byte
Caratteri visibili: 10

Nota: in UTF-16 sarebbero 20 byte (10×2)
in UTF-32 sarebbero 40 byte (10×4)

Esercizio 5: Numero vs Carattere (20 min)

5.1 Qual è la differenza in memoria tra:

```
int x = 5;
char c = '5';
```

Soluzione:
int x = 5:
- Tipo: intero
- Valore: 5
- In memoria (4 byte, little-endian):
  0x00000005 = 00000000 00000000 00000000 00000101
- Uso: calcoli matematici

char c = '5':
- Tipo: carattere
- Valore ASCII: 53
- In memoria (1 byte):
  0x35 = 00110101
- Uso: rappresentazione testuale

NON sono intercambiabili!
x + 2 = 7
c + 2 = 55 (ASCII di '7'... no, sarebbe 55, che è '7'? No!)

Corretto:
(c - '0') + 2 = 5 + 2 = 7

5.2 Cosa stampa il seguente codice?

```
char c = 'A';
printf("%c\n", c);
printf("%d\n", c);
printf("%c\n", c + 1);
printf("%d\n", c + 32);
```

```
Soluzione:
char c = 'A';  // c = 65

printf("%c\n", c);      // Stampa: A (come carattere)
printf("%d\n", c);      // Stampa: 65 (come numero)
printf("%c\n", c + 1);  // Stampa: B (65+1=66='B')
printf("%d\n", c + 32); // Stampa: 97 (65+32=97, codice di 'a')
```

5.3 Scrivi pseudocodice per convertire una stringa di cifre in numero:

Input: “3456”
Output: 3456

```
Soluzione:
INIZIO
   LEGGI stringa
   numero ← 0

   PER ogni carattere c in stringa FARE
      SE c è una cifra ('0' a '9') ALLORA
         cifra ← valore_ASCII(c) - valore_ASCII('0')
         numero ← numero × 10 + cifra
      ALTRIMENTI
         ERRORE "Carattere non valido"
      FINE SE
   FINE PER

   SCRIVI numero
FINE

Esempio traccia con "3456":
Inizio: numero = 0
c='3': cifra=51-48=3, numero=0×10+3=3
c='4': cifra=52-48=4, numero=3×10+4=34
c='5': cifra=53-48=5, numero=34×10+5=345
```

```
c='6': cifra=54-48=6, numero=345×10+6=3456
Fine: numero = 3456
```

ESERCIZI AUTONOMI

Esercizio 6: Problemi Misti

6.1 Una temperatura viene memorizzata come float IEEE 754. Il valore in memoria è:

```
0 10000010 01000000000000000000000
```

Qual è la temperatura in gradi Celsius?

6.2 Scrivi in binario (virgola fissa 8.8):

- 15.75
- 0.5
- 127.99609375

6.3 Codifica la stringa “C Programming!” in:

- ASCII (decimale)
- ASCII (esadecimale)
- UTF-8 (esadecimale)

6.4 Determina la dimensione totale in byte delle seguenti stringhe in UTF-8:

- “Caffè”
- “Привет” (russo: “Ciao”)
- “こんにちは” (giapponese: “Ciao”)
- “Hello 🌍!”

ESERCIZI DA SVOLGERE A CASA

Homework 1: Crea una tabella con i numeri da 0 a 1 con passo 0.125 (0, 0.125, 0.250, …, 1.0) e converti ognuno in:

- Binario (parte decimale)
- IEEE 754 singola precisione (rappresentazione schematica)

Homework 2: Scrivi un algoritmo (pseudocodice o flow chart) che:

- Legge una stringa contenente solo cifre
- Converte la stringa nel corrispondente numero intero
- Gestisce il caso di stringhe vuote o con caratteri non validi

Homework 3: Analizza il problema del confronto di float:

- Perché `0.1 + 0.2 == 0.3` è falso?
- Come si dovrebbe fare il confronto correttamente?
- Scrivi una funzione (pseudocodice) per confrontare due float con tolleranza

Homework 4: Ricerca e documenta:

- Cos'è l'ULP (Unit in the Last Place)?
- Cosa significa "denormalized number" in IEEE 754?
- Casi pratici dove float causa problemi (finanza, fisica, ecc.)

Homework 5: Crea un programma (pseudocodice) che:

- Legge una stringa UTF-8
- Conta quanti byte occupa in memoria
- Conta quanti caratteri visibili contiene
- Distingue tra caratteri ASCII e non-ASCII

Homework 6: Esercizio di conversione completo:
Dato il numero -25.625_{10}:
a) Convertilo in binario (parte intera e decimale)
b) Rappresentalo in virgola fissa 16.16
c) Rappresentalo in IEEE 754 (32 bit)
d) Verifica entrambe le rappresentazioni riconvertendo in decimale

---

APPROFONDIMENTI

Il Problema dello 0.1 + 0.2

Questo è uno dei problemi più famosi del floating point:

```
>>> 0.1 + 0.2
0.30000000000000004
```

Perché succede?

0.1 in binario è periodico:

$$0.1_{10} = 0.0001100110011001100110011..._2 \text{ (infinito)}$$

Il computer deve troncare, quindi memorizza un'approssimazione:

```
0.1 ≈ 0.1000000000000000055511151231257827...
0.2 ≈ 0.2000000000000000011102230246251565 4...
Somma ≈ 0.3000000000000000444089209850062...
```

Conseguenze:

- Mai usare `==` per confrontare float/double
- Problemi nei cicli: `for (float x = 0; x != 1.0; x += 0.1)` → infinito!
- Errori accumulati nei calcoli lunghi

Soluzioni:

- Usare epsilon per confronti: `fabs(a - b) < 1e-9`
- Librerie di precisione arbitraria (GMP, MPFR)
- Aritmetica decimale per finanza (Java BigDecimal, Python Decimal)

Floating Point in Hardware

Le CPU moderne hanno unità dedicate per floating point: **FPU** (Floating Point Unit).

Ottimizzazioni:

- Istruzioni SIMD: SSE, AVX (calcoli paralleli)
- Fusione operazioni: FMA (Fused Multiply-Add: a×b+c in una istruzione)
- Pipeline specializzate

Modalità di arrotondamento IEEE 754:

1. Round to nearest (default)
2. Round toward zero
3. Round toward +∞
4. Round toward -∞

Storia di Unicode

Timeline:

- 1987: Joe Becker (Xerox) propone Unicode
- 1991: Unicode 1.0 (7,161 caratteri)
- 1996: UTF-8 creato da Ken Thompson e Rob Pike
- 2001: UTF-8 diventa dominante sul web
- 2010: Emoji aggiunte a Unicode
- 2022: Unicode 15.0 (149,186 caratteri)

Curiosità:

- Il code point più alto: U+10FFFF
- Esistono blocchi riservati per uso privato
- Emoji con toni della pelle usano “modifier”
- Alcune emoji sono sequenze: 👨‍👩‍👧 = 7 code point!

Endianness nei Floating Point

Anche i float hanno problemi di endianness quando trasferiti tra sistemi!

Esempio: 12.375 in IEEE 754

```
Big-endian:    41 46 00 00
Little-endian: 00 00 46 41
```

Per questo esistono formati di serializzazione neutrali (JSON, Protocol Buffers).

Confronto Prestazioni: Virgola Fissa vs Mobile

Virgola Fissa:

- Più veloce (operazioni intere)
- Usata in DSP, embedded systems
- Librerie: libfixmath

Virgola Mobile:

- Più flessibile
- Standard moderno
- Hardware dedicato (FPU)

Benchmark tipico (milioni di operazioni/sec):

```
Integer:        1000
Fixed-point:    800
Float (FPU):    500
Float (software): 50
```

Charset Storici

EBCDIC (IBM):

- Usato su mainframe IBM
- Incompatibile con ASCII!
- Ancora in uso in sistemi legacy

ISO 8859 (15 varianti):

- ISO 8859-1 (Latin-1): Europa occidentale
- ISO 8859-2 (Latin-2): Europa centrale
- ISO 8859-5: Cirillico
- Problema: incompatibili tra loro!

Windows-1252:

- “ANSI” di Windows
- Estensione non standard di ISO 8859-1
- Aggiunge caratteri 128-159

Il Futuro: Oltre IEEE 754?

Problemi dell’IEEE 754:

- Errori di arrotondamento inevitabili
- NaN e infiniti complicano la semantica
- Confronti non intuitivi

Alternative proposte:

- **Posit** (John Gustafson, 2017): maggiore precisione dinamica
- **Unums**: numeri con incertezza incorporata
- **Decimale binario**: IEEE 754-2008 include float decimali

Nessuna ha ancora sostituito lo standard consolidato.

RIEPILOGO FORMULE E CONCETTI

Conversione Binario con Virgola → Decimale

$$N.M_2 = \Sigma(\text{bit} \times 2^{\text{posizione}})$$

Esempio: 101.11_2

$$= 1\times2^2 + 0\times2^1 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-2}$$
$$= 4 + 0 + 1 + 0.5 + 0.25$$
$$= 5.75_{10}$$

Conversione Decimale → Binario (Parte Decimale)

Moltiplica ripetutamente per 2
Prendi la parte intera (0 o 1)
Continua con la parte decimale
Leggi dall'alto verso il basso

IEEE 754 Formula

Valore = (-1)^S × (1.M) × 2^(E-bias)

Float (32 bit):
- S: 1 bit
- E: 8 bit (bias = 127)
- M: 23 bit (+ 1 bit nascosto)
- Range: $\pm 1.18 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$

Double (64 bit):
- S: 1 bit
- E: 11 bit (bias = 1023)
- M: 52 bit (+ 1 bit nascosto)
- Range: $\pm 2.23 \times 10^{-308}$ a $\pm 1.8 \times 10^{308}$

ASCII - Relazioni Chiave

Maiuscola ↔ Minuscola: differenza = 32 (0x20)
'A' = 65, 'a' = 97
'Z' = 90, 'z' = 122

Cifra → Numero: sottrai 48 (o '0')
'0' = 48, '9' = 57
'5' - '0' = 53 - 48 = 5

```
Numero → Cifra: aggiungi 48 (o '0')
5 + '0' = 5 + 48 = 53 = '5'
```

UTF-8 Schema

```
1 byte:  0xxxxxxx                    (0-127)
2 byte:  110xxxxx 10xxxxxx           (128-2047)
3 byte:  1110xxxx 10xxxxxx 10xxxxxx  (2048-65535)
4 byte:  11110xxx 10xxxxxx 10xxxxxx 10xxxxxx (65536+)
```

TABELLE DI RIFERIMENTO RAPIDO

Potenze di 2 (Decimali)

```
2⁻⁸ = 0.00390625    2⁰ = 1
2⁻⁷ = 0.0078125     2¹ = 2
2⁻⁶ = 0.015625      2² = 4
2⁻⁵ = 0.03125       2³ = 8
2⁻⁴ = 0.0625        2⁴ = 16
2⁻³ = 0.125         2⁵ = 32
2⁻² = 0.25          2⁶ = 64
2⁻¹ = 0.5           2⁷ = 128
```

Frazioni Binarie Comuni

```
Decimale | Binario  | Frazione
---------|----------|----------
0.5      | .1       | 1/2
0.25     | .01      | 1/4
0.125    | .001     | 1/8
0.75     | .11      | 3/4
```

```
0.625   | .101    | 5/8
0.375   | .011    | 3/8
0.875   | .111    | 7/8
```

ASCII Caratteri Speciali

```
Dec | Hex | Char | Nome
----|-----|------|-------------
0   | 00  | NUL  | Null
9   | 09  | TAB  | Tab
10  | 0A  | LF   | Line Feed
13  | 0D  | CR   | Carriage Return
32  | 20  | ' '  | Spazio
33  | 21  | !    | Punto esclamativo
48  | 30  | 0    | Zero
65  | 41  | A    | A maiuscola
97  | 61  | a    | a minuscola
127 | 7F  | DEL  | Delete
```

Valori Speciali IEEE 754

```
Caso          | Esponente | Mantissa | Esempio (32-bit)
--------------|-----------|----------|------------------
Zero          | Tutti 0   | 0        | 0x00000000
Denormalized  | Tutti 0   | ≠ 0      | 0x00000001
Normalized    | 1-254     | qualsiasi| 0x3F800000 (1.0)
+Infinito     | Tutti 1   | 0        | 0x7F800000
-Infinito     | Tutti 1   | 0        | 0xFF800000
NaN           | Tutti 1   | ≠ 0      | 0x7FC00000
```

PROBLEMI COMUNI E SOLUZIONI

Problema 1: Confronto Float

```
// ❌ MALE
if (x == 0.1) { ... }

// ✅ BENE
#define EPSILON 1e-9
if (fabs(x - 0.1) < EPSILON) { ... }
```

Problema 2: Cicli con Float

```
// ❌ MALE (può non terminare!)
for (float x = 0.0; x != 1.0; x += 0.1) {
    printf("%f\n", x);
}

// ✅ BENE
for (float x = 0.0; x < 1.0; x += 0.1) {
    printf("%f\n", x);
}

// ✅ ANCORA MEGLIO
for (int i = 0; i < 10; i++) {
    float x = i * 0.1f;
    printf("%f\n", x);
}
```

Problema 3: Perdita di Precisione

```
// ❌ Problema
float grande = 1e20f;
float piccolo = 1.0f;
float somma = grande + piccolo;  // somma == grande!

// ✅ Soluzione: ordina le operazioni
// Somma prima i valori piccoli
```

Problema 4: Conversione Char/Int

```
char c = '5';

// ❌ MALE
int n = (int)c;  // n = 53 (ASCII!)

// ✅ BENE
int n = c - '0';  // n = 5
```

Problema 5: Stringhe vs Numeri

```
// ❌ Confusione
char str[] = "123";
int x = str;  // ERRORE! str è un puntatore

// ✅ BENE
char str[] = "123";
int x = atoi(str);  // x = 123
```

ESERCIZI DI RIEPILOGO COMPLETO

Esercizio Integrato 1

Dato il numero 13.625_{10}:
a) Convertilo in binario (parte intera + decimale)
b) Rappresentalo in virgola fissa 8.8
c) Rappresentalo in IEEE 754 (32 bit)
d) Quanto spazio occuperebbe la stringa “13.625” in:

- ASCII
- UTF-8

Soluzioni:

a) Binario:
$13_{10} = 1101_2$
$0.625_{10} = .101_2$
Risultato: 1101.101_2

b) Virgola fissa 8.8:
Parte intera: 00001101 (8 bit)
Parte decimale: 10100000 (8 bit, .101 + zeri)
Totale: 00001101.10100000

c) IEEE 754:
Normalizzato: 1.101101×2^3
S = 0 (positivo)
E = 3 + 127 = 130 = 10000010_2
M = 10110100000000000000000 (dopo il bit nascosto)
Risultato: 0 10000010 10110100000000000000000

d) Stringa "13.625":
6 caratteri visibili + 1 terminatore '\0' = 7 byte
Tutti i caratteri sono ASCII (< 128):

ASCII: 7 byte
UTF-8: 7 byte (identico per caratteri ASCII)

Esercizio Integrato 2

Analizza la seguente sequenza di byte in memoria:

```
48 65 6C 6C 6F 20 F0 9F 98 80 21
```

Determina:
a) È una stringa ASCII valida?
b) È una stringa UTF-8 valida?
c) Cosa rappresenta?
d) Quanti caratteri visibili contiene?
e) Quanti byte occupa?

Soluzioni:

a) NO - I byte F0 9F 98 80 sono > 127 (non ASCII)

b) SÌ - È UTF-8 valido

c) Decodifica:
48 = 'H'
65 = 'e'
6C = 'l'
6C = 'l'
6F = 'o'
20 = ' ' (spazio)
F0 9F 98 80 = 😀 (emoji, 4 byte UTF-8)
21 = '!'

Stringa: "Hello 😀!"

d) 8 caratteri visibili (l'emoji conta come 1 carattere)

e) 11 byte totali (7 ASCII + 4 emoji)

Esercizio Integrato 3

Un programma deve memorizzare temperature da -50.0°C a +150.0°C con precisione di 0.1°C.

1. Conviene usare virgola fissa o mobile? Perché?
2. Se si usa virgola fissa, quale formato scegliere?
3. Se si usa float, quanti bit di mantissa sono necessari per la precisione richiesta?
4. Quale occupa meno memoria?

Soluzioni:

a) Virgola FISSA è migliore per questo caso:
- Range limitato e noto
- Precisione fissa richiesta
- Più veloce
- Più predicibile

b) Formato virgola fissa:
Range: -50.0 a +150.0 → serve segno
Max valore assoluto: 150.0
150 richiede 8 bit (256 > 150)
Precisione 0.1 → serve 1 cifra decimale
0.1 = 1/10, ma in binario serve più precisione

Soluzione pratica:
- Moltiplica tutto per 10: range -500 a +1500
- Memorizza come intero signed a 16 bit (-32768 a +32767)
- Quando leggi, dividi per 10.0
- Spazio: 2 byte

c) Float IEEE 754:
Mantissa: 23 bit

Precisione: ~7 cifre decimali
Sufficiente per 0.1°C su range 200°C
Spazio: 4 byte

d) Virgola fissa: 2 byte
Float: 4 byte
→ Virgola fissa più efficiente (50% spazio in meno)

PUNTI CHIAVE DELLA LEZIONE

✓ **Virgola fissa**: range e precisione predefiniti, veloce, semplice
✓ **Virgola mobile** (IEEE 754): flessibile, standard universale
✓ **Non tutti i decimali** hanno rappresentazione binaria finita (es. 0.1)
✓ **Mai confrontare float** con `==`, usare epsilon
✓ **ASCII**: 7 bit, 128 caratteri, solo inglese base
✓ **UTF-8**: lunghezza variabile, compatibile ASCII, universale
✓ **Numero ≠ Carattere**: '5' (codice 53) ≠ 5 (valore numerico)
✓ **Conversione char→int**: sottrai '0' (48)
✓ **Maiuscola/minuscola**: differenza di 32 (bit 5)
✓ **Float problematici**: arrotondamenti, confronti, accumulo errori

---

PREPARAZIONE PER LA PROSSIMA LEZIONE

Nella prossima lezione studieremo:

- **Architettura del computer**: Von Neumann
- **CPU**: ALU, Control Unit, registri
- **Ciclo fetch-decode-execute**: come la CPU esegue istruzioni
- **Memoria**: RAM, cache, gerarchia
- **Istruzioni a basso livello**: introduzione all'assembly

Porta con te:

- Esercizi homework completati
- Calcolatrice programmabile (se disponibile)
- Domande su floating point o charset
- Tabelle di conversione create

Suggerimento: Installa un emulatore di CPU semplice (es. CPU Simulator, Little Man Computer) per visualizzare il funzionamento interno.

GLOSSARIO TECNICO

- **Fixed Point**: Virgola fissa, posizione decimale predefinita
- **Floating Point**: Virgola mobile, esponente variabile
- **IEEE 754**: Standard per numeri in virgola mobile
- **Mantissa/Significand**: Cifre significative di un numero float
- **Esponente**: Potenza nella notazione scientifica
- **Bias**: Offset per esponente (127 float, 1023 double)
- **Bit nascosto**: Il “1.” implicito nella mantissa IEEE 754
- **NaN**: Not a Number, risultato di operazioni invalide
- **Denormalized**: Numeri molto piccoli vicini a zero
- **ULP**: Unit in Last Place, precisione minima
- **ASCII**: American Standard Code for Information Interchange
- **Code Point**: Numero assegnato a un carattere in Unicode
- **UTF-8**: Codifica Unicode lunghezza variabile (1-4 byte)
- **BOM**: Byte Order Mark, intestazione file per indicare encoding
- **Endianness**: Ordine dei byte in memoria (big/little-endian)

RISORSE AGGIUNTIVE

Tool Online Utili

Conversione Float:

- https://www.h-schmidt.net/FloatConverter/IEEE754.html
- Visualizza IEEE 754 in dettaglio

Tabelle ASCII:

- https://www.asciitable.com/
- https://ascii.cl/

Unicode Explorer:

- https://unicode-table.com/
- Cerca code point e visualizza caratteri

Conversioni Numeriche:

- https://www.rapidtables.com/convert/number/
- Binario, ottale, decimale, esadecimale

Letture Consigliate

Paper fondamentali:

- “What Every Computer Scientist Should Know About Floating-Point Arithmetic” (David Goldberg, 1991)
- “The Unicode Standard” (Unicode Consortium)

Libri:

- “Computer Systems: A Programmer’s Perspective” (Bryant & O’Hallaron) - Capitolo 2
- “The Art of Computer Programming, Vol 2” (Knuth) - Aritmetica

Esercizi Interattivi

- **Float Toy**: http://evanw.github.io/float-toy/
  Visualizza IEEE 754 interattivamente

- **UTF-8 Tool**: https://mothereff.in/utf-8
  Analizza stringhe UTF-8

DOMANDE DI AUTOVALUTAZIONE

Rispondi senza guardare gli appunti, poi controlla:

1. Perché 0.1 non ha rappresentazione binaria finita?
2. Qual è la differenza principale tra float e double?
3. Cosa significa il bit nascosto in IEEE 754?
4. Quando un float diventa NaN?
5. Qual è il codice ASCII di ‘A’? E di ‘a’?
6. Come converto velocemente maiuscola in minuscola?
7. Quanti byte usa UTF-8 per ‘è’? E per ‘😬’?
8. Qual è la differenza tra numero 5 e carattere ‘5’?
9. Come si converte ‘7’ nel numero 7?
10. Perché `x == 0.3` è pericoloso con float?

Risposte rapide:

1. 0.1_{10} = 0.00011001100…$_2$ (periodico)
2. float: 32 bit, 7 cifre; double: 64 bit, 15 cifre
3. Il “1.” della mantissa non viene memorizzato (guadagno 1 bit)
4. Operazioni invalide: 0/0, ∞-∞, √(-1)
5. ‘A’ = 65; ‘a’ = 97
6. Somma/sottrai 32, o usa OR 0x20 / AND ~0x20
7. ‘è’ = 2 byte; ‘😬’ = 4 byte
8. 5 = valore numerico (0x05); ‘5’ = codice ASCII 53 (0x35)
9. ‘7’ - ‘0’ = 55 - 48 = 7
10. Errori di arrotondamento rendono confronti imprecisi

PROGETTO FINALE LEZIONE 4

Mini-Progetto: Analizzatore di Stringhe e Numeri

Scrivi uno pseudocodice per un programma che:

Input: Una stringa qualsiasi (può contenere numeri, lettere, spazi, UTF-8)

Output:

1. Numero totale di caratteri
2. Numero di byte occupati
3. Quanti caratteri ASCII
4. Quanti caratteri non-ASCII
5. Quante lettere maiuscole
6. Quante lettere minuscole
7. Quante cifre numeriche
8. Se la stringa contiene solo cifre, convertila in numero

Esempio:

Input: "Hello123 😬"

Output:
- Caratteri totali: 10
- Byte occupati: 14 (7 ASCII + 1 spazio + 4 emoji + 1 terminatore)
- Caratteri ASCII: 9
- Caratteri non-ASCII: 1 (emoji)
- Maiuscole: 1 ('H')
- Minuscole: 4 ('e','l','l','o')
- Cifre: 3 ('1','2','3')
- Non è un numero puro

Input: "42"
Output:
- Caratteri totali: 2
- Byte occupati: 3
- Caratteri ASCII: 2
- Caratteri non-ASCII: 0
- Maiuscole: 0
- Minuscole: 0
- Cifre: 2
- Valore numerico: 42

Estensione (difficile):
Aggiungi la capacità di rilevare se una stringa è UTF-8 valida analizzando i pattern dei byte.

Ottimo lavoro! Ora comprendi come i computer rappresentano numeri e testo! 🎯

Nella prossima lezione scenderemo ancora più in profondità: vedremo come la CPU esegue fisicamente le istruzioni e come funziona la memoria.